Gardener F5 Application Load Balancer Extension

Part 2 - Component Architecture and Internal Desired-State Model

Architecture and Low-Level Design
Version 1.0 - Design Baseline
17 July 2026

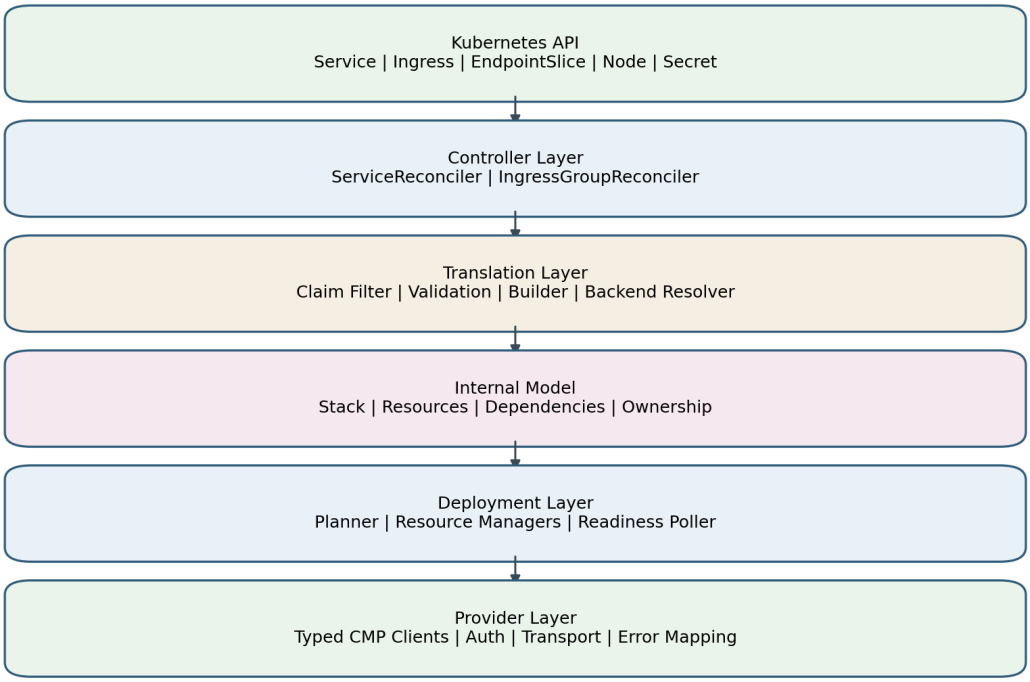

Figure 2-1. Target layered architecture

1. Purpose and Position in the Design Set

This part defines the internal architecture that translates Kubernetes networking intent into CMP/F5 resources. It follows Part 1, which established the architecture basis and provider capability mapping. This document fixes the component boundaries, internal resource model, dependency rules, ownership contract, backend-resolution contract, and package-level interfaces that Parts 3 through 7 will implement in detail.

The AWS Load Balancer Controller is used as an architectural reference, specifically for its thin reconcilers, dedicated model builders, stack-based desired state, resource-specific deployment managers, finalizer handling, and provider tracking. The design does not reproduce AWS-specific resources or behavior. CMP OpenAPI remains the authoritative provider capability contract. The existing Gardener F5 code is treated only as evidence for authentication, endpoint usage, request/response details, and integration constraints.

1.1 Fixed architectural outcome

```
Kubernetes objects
    -> thin reconcilers
    -> validation and backend resolution
    -> typed desired-state stack
    -> resource-specific deployment managers
    -> typed CMP clients
    -> CMP/F5
```

- Controllers do not construct CMP payloads.
- Builders do not mutate CMP.
- Deployment managers do not interpret Kubernetes annotations.
- CMP clients do not contain Kubernetes business rules.
- Every managed provider object carries verifiable ownership metadata.
- Correctness is based on reconciliation from durable Kubernetes and CMP state, never on controller memory.

1.2 Scope of Part 2

Included	Deferred to later parts
Component boundaries; desired/observed model; backend resolution; ownership; finalizer architecture; deployment orchestration; package design; Go interfaces	Service-specific semantics and algorithms - Part 3
Shared foundations for Service and Ingress	Ingress rules, grouping, TLS details - Part 4
Provider-manager contracts and readiness gates	Full endpoint-by-endpoint client behavior - Part 5
Representative reconciliation sequence	All runtime sequences and state machines - Part 6
Implementation slices and package acceptance criteria	Complete migration and test execution plan - Part 7

2. Architectural Evidence and Adaptation Rules

2.1 Patterns adopted from AWS Load Balancer Controller

AWS LBC pattern	Reason it is proven	F5/CMP adaptation
Separate Service reconciler and Ingress-group reconciler	Different Kubernetes ownership, events, grouping, status and deletion semantics	Keep independent controllers/builders; share model, deployer, resolver and clients
ModelBuilder.Build returns a complete stack	Provider operations are separated from Kubernetes interpretation	Build an F5 Stack with LBService, VIP, VirtualServers, Pools, Members, Monitors, Rules and Certificates
StackDeployer deploys resource graph	Resource ordering and provider reconciliation are centralized	CMP StackDeployer coordinates typed managers and asynchronous readiness
Resource-specific managers/synthesizers	Each provider resource has focused diff and lifecycle logic	LBServiceManager, VIPManager, VirtualServerManager, PoolManager, MemberManager, MonitorManager, RoutingRuleManager, CertificateManager
Finalizers around external-resource cleanup	Kubernetes object cannot vanish before cloud cleanup	Service finalizer and group-aware Ingress finalizers
Tracking metadata/tags	Safe discovery, adoption and deletion	CMP labels where supported plus deterministic ownership envelope
Event handlers map related-resource changes to owners	Endpoint, Service, Secret and class changes enqueue correct reconciliations	Explicit watch/index graph for Service and Ingress

2.2 Patterns not copied blindly

AWS-specific element	Decision
ELBv2 LoadBalancer/Listener/TargetGroup schemas	Replace with CMP LB Service/VIP/Virtual Server/Pool/Member schemas
Security-group synthesis	Not part of the core F5 object model; integrate only where Airtel network policy requires it
AWS subnet/VPC discovery rules	Use Gardener Shoot configuration and CMP network identifiers
AWS target registration modes	Initial target mode is node/VM plus NodePort because CMP members require VM/network identity
AWS tagging semantics	Use CMP labels/description fields and ownership verification compatible with Swagger
ALB listener action JSON	Translate Ingress routing semantics to CMP routing-rule API and default-pool operation

3. Target Component Architecture

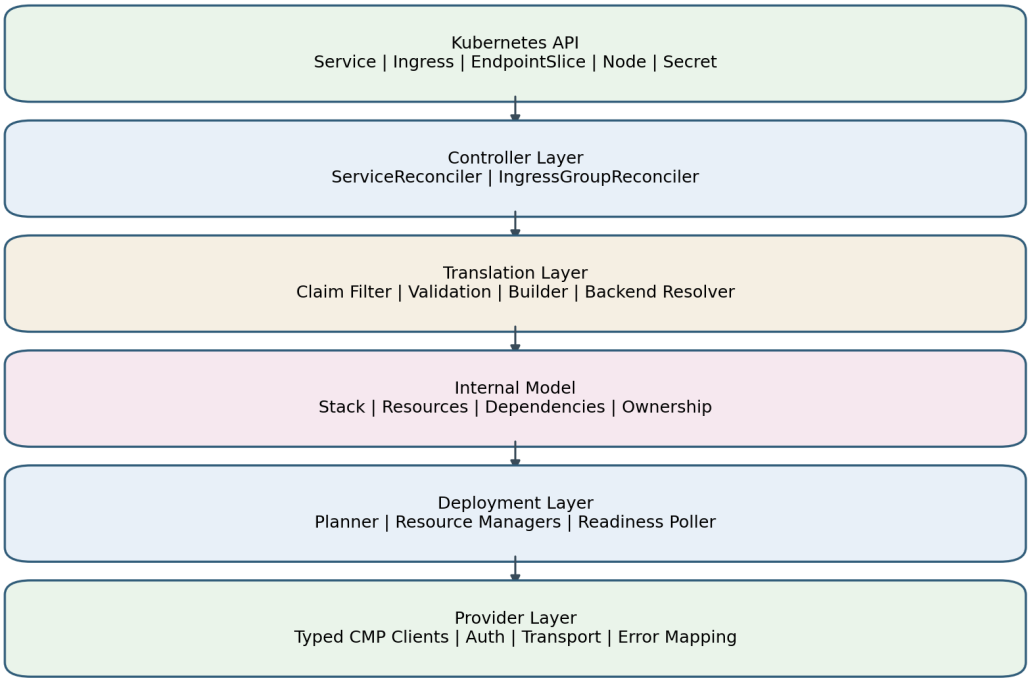

Figure 2-2. Layered component architecture

Each layer owns one type of decision. Downward dependencies are permitted; upward dependencies are prohibited. Cross-layer shortcuts, such as a controller importing HTTP transport types or a CMP client reading Kubernetes annotations, are architecture violations.

3.1 Component inventory

Component	Primary responsibility	Must not do
Controller Manager	Wiring, leader election, health, metrics, configuration, client construction	Load-balancer business logic
ServiceReconciler	Claim Service, coordinate finalizer/build/deploy/status	Construct provider payloads
IngressGroupReconciler	Load group, coordinate group finalizers/build/deploy/status	Implement route translation itself
Claim Filter	Determine whether an object belongs to this controller	Mutate objects or provider state
Validator	Reject unsupported or inconsistent Kubernetes intent	Perform provider writes
Model Builder	Create complete deterministic desired-state graph	Read or mutate actual CMP state
Backend Resolver	Resolve endpoints/nodes to authoritative CMP identities	Invent resource IDs or backend_port_id
Stack Deployer	Order managers, maintain transaction context, aggregate result	Parse annotations
Resource Manager	Reconcile one resource type	Control unrelated resource types
Typed CMP Client	API paths, auth, encoding, decoding, errors, timeouts	Kubernetes reconciliation policy
Ownership Provider	Generate and verify ownership envelope	Infer ownership from name alone
Status Writer	Patch Service/Ingress status and expose stable observations	Provision provider resources

4. Controller Boundaries and Watch Graph

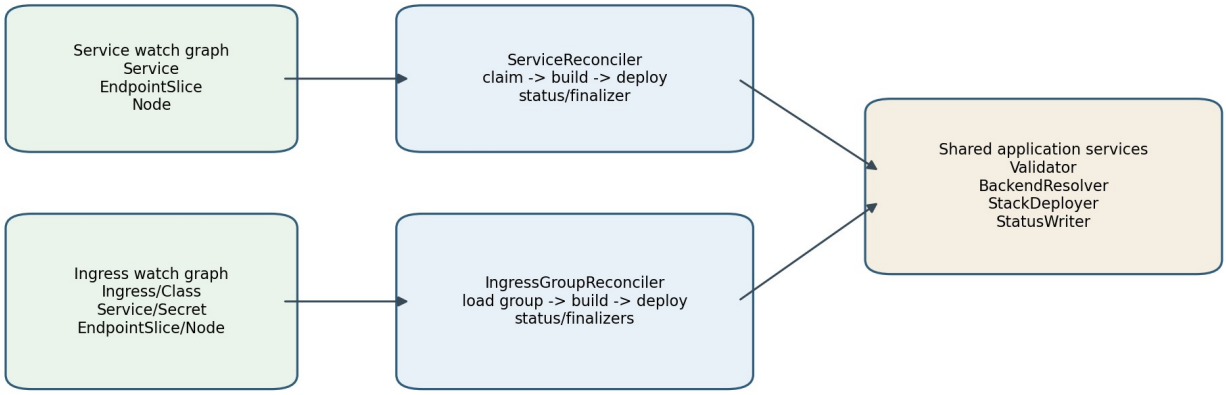

Figure 2-3. Controller boundaries and shared application services

4.1 Service controller contract

- Owns Service resources selected by loadBalancerClass and/or the supported compatibility claim rule.
- Watches Service directly and maps EndpointSlice and Node changes back to affected Services.
- Adds the Service finalizer before creating external resources.
- Builds exactly one logical stack per Service unless an explicit shared-frontend feature is enabled.
- Publishes Service.status.loadBalancer only after the selected readiness gate passes.

4.2 Ingress controller contract

- Owns Ingress resources selected through the F5 IngressClass.
- Reconciles an Ingress group rather than an isolated Ingress when grouping is enabled.
- Watches Ingress, IngressClass, IngressClassParameters if introduced, referenced Services, Secrets, EndpointSlices and Nodes.
- Builds one stack for the active group and treats inactive members as cleanup inputs.
- Uses group-aware finalizer management so a shared frontend is not deleted while active members remain.

4.3 Event-to-owner indexing

Event source	Service enqueue rule	Ingress enqueue rule
Service	self	Ingresses referencing Service
EndpointSlice	Service label kubernetes.io/service-name	Ingresses referencing owning Service
Node	all managed node-target Services, with debouncing/index optimization	all groups whose backends use node target mode
Ingress	none	group ID of Ingress
Secret	none	groups referencing TLS Secret
IngressClass	none	Ingresses using class

5. Reconciliation Pipeline

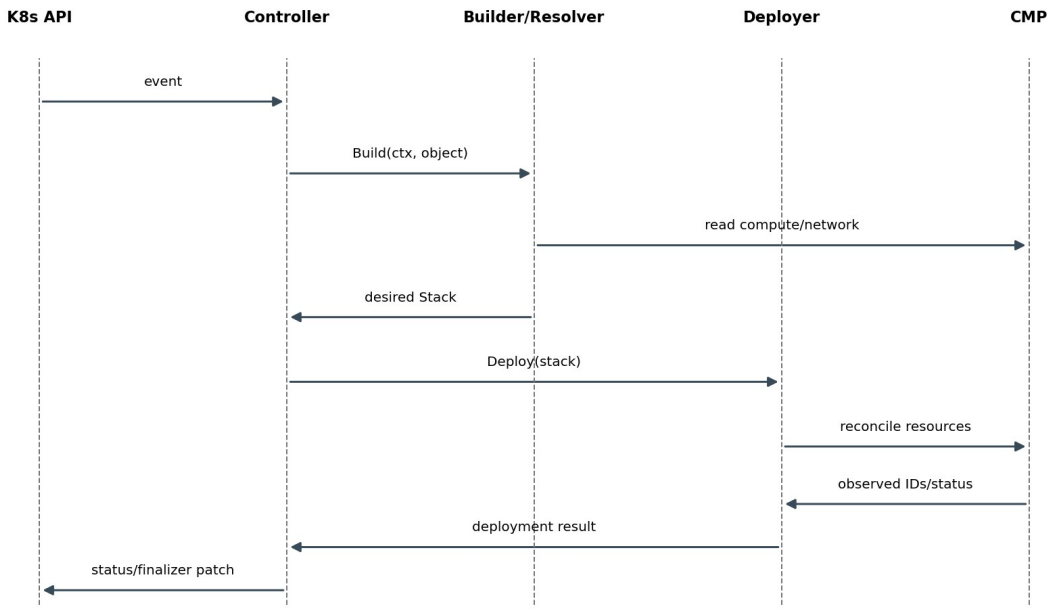

Figure 2-4. Representative reconciliation sequence

5.1 Common orchestration algorithm

1. Fetch the latest Kubernetes owner or group.
2. Evaluate claim/ownership.
3. If deleting or no longer claimed, build an empty desired stack and clean up owned CMP resources.
4. Validate configuration and provider capability constraints.
5. Add finalizer before first external create.
6. Resolve backends and supporting identities.
7. Build a complete deterministic desired stack.
8. Discover observed CMP inventory using ownership and stored IDs.
9. Produce and execute a dependency-ordered reconciliation plan.
10. Poll only resources whose API is asynchronous and whose readiness is required.
11. Patch Kubernetes status and emit events/metrics.
12. Remove finalizer only after required owned resources are absent.

5.2 Error ownership

Error source	Classification owner	Controller behavior
Unsupported Service/Ingress field combination	Validator	Permanent until object changes; Event; no fast retry
Node has no InternalIP	Backend Resolver	Dependency not ready; bounded requeue
CMP compute/network lookup timeout	CMP client + resolver	Retryable transport error
CMP 401/403	CMP client	Authentication/authorization; slow retry and alert
CMP 409 during create	Resource manager	Rediscover and verify ownership/idempotency
CMP provisioning still in progress	Resource manager/readiness poller	RequeueAfter without treating as failure
Status patch conflict	Status writer	Refetch and retry patch

6. Internal Desired-State Model

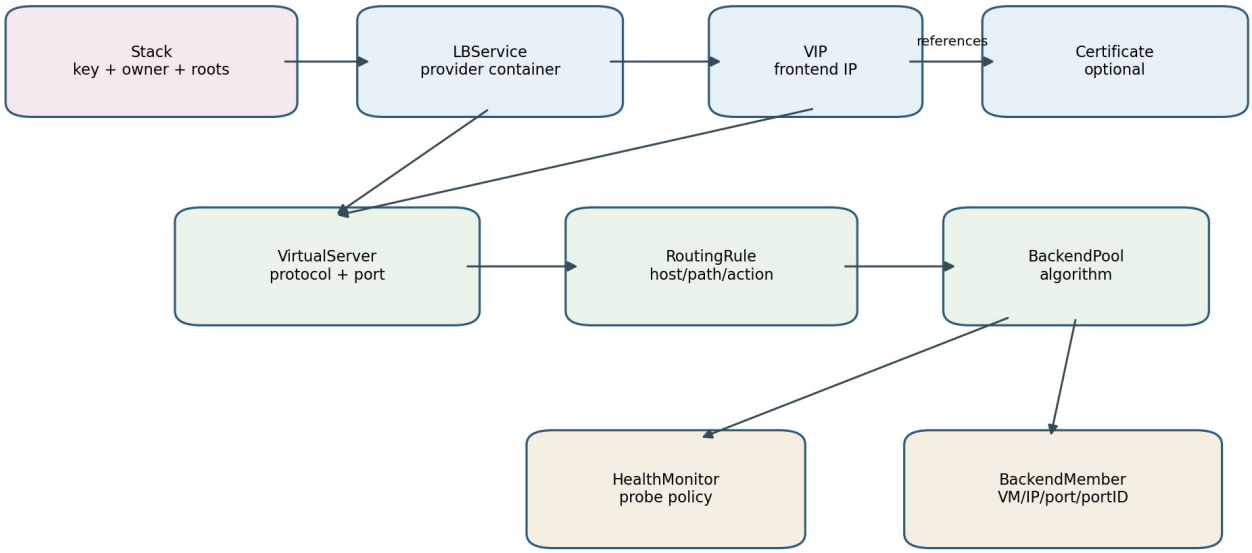

Figure 2-5. Internal desired-state resource graph

The internal model is an in-memory typed graph. It is not a CRD and is not persisted as a second source of truth. It captures the complete desired provider configuration for one reconciliation key and contains enough metadata for deterministic deployment and cleanup.

6.1 Core abstractions

Type	Purpose	Key invariants
Stack	Root graph, stack key, owner, resource registry and dependency edges	Stable key; no duplicate logical IDs; acyclic dependencies
ResourceMeta	Logical ID, external ID if observed, ownership, dependencies	Logical ID deterministic; external ID never fabricated
LBService	CMP service container and network/flavor context	Exactly one root per non-shared stack
VIP	Allocated or requested frontend address	References LBService; published only when ready
VirtualServer	Protocol/port frontend listener equivalent	Unique frontend tuple within stack
RoutingRule	Host/path match and pool action	Ordered deterministically; references existing pool
BackendPool	Backend group, balancing algorithm and default status	Unique per backend identity and routing semantics
HealthMonitor	Probe definition attached to pool	Protocol compatible with pool/backend
BackendMember	Authoritative target identity and service port	resource_id, resource_ip and backend_port_id resolved from CMP
Certificate	Certificate reference/attachment model	Secret material not placed in logs or logical IDs

6.2 Proposed Go model

```
type Stack struct {
    Key       StackKey
```

```
    Owner     Ownership
    Resources ResourceRegistry
    Edges     []Dependency
}

type ResourceMeta struct {
    LogicalID    string
    ExternalID   string // observed only
    Role         ResourceRole
    Ownership    Ownership
    DependsOn    []string
}

type LBService struct { Meta ResourceMeta; Spec LBServiceSpec }
type VIP struct { Meta ResourceMeta; Spec VIPSpec }
type VirtualServer struct { Meta ResourceMeta; Spec VirtualServerSpec }
type BackendPool struct { Meta ResourceMeta; Spec BackendPoolSpec }
type BackendMember struct { Meta ResourceMeta; Spec BackendMemberSpec }
```

7. Resource Specifications and Identity

7.1 LBServiceSpec

Field	Source	Rule
Name	NamingProvider	Deterministic and CMP length-safe
Description	OwnershipProvider	Human-readable ownership envelope
FlavorID	Controller config / supported annotation	Validated against permitted flavor set
NetworkID	Shoot/provider config	Required and immutable unless replacement is explicitly supported
VPCID/VPCName	Shoot/provider config	Must match backend network context
Labels	OwnershipProvider	Canonical managed-by, cluster and source identity

7.2 VIPSpec

Field	Meaning
LBServiceRef	Logical reference to parent
RequestedAddress	Optional only if API and policy allow static address
AddressFamily	IPv4 initially; dual stack deferred until verified
Scheme/Visibility	Provider-supported visibility policy
Ownership	Inherited plus role=vip

7.3 VirtualServerSpec

Field	Meaning
VIPRef	Frontend address reference
Protocol	TCP/HTTP/HTTPS according to validated mapping
Port	Frontend service/listener port
Persistence	CMP-supported persistence policy
DefaultPoolRef	Pool used when no rule matches
CertificateRefs	TLS attachments
SourceCIDRs	Only if CMP endpoint supports equivalent semantics
ConnectionPolicy	Only supported CMP attributes

7.4 BackendPool and Member specifications

Type	Critical fields
BackendPoolSpec	VirtualServerRef, algorithm, default flag, monitor reference, backend key
BackendMemberSpec	resource_id, resource_type, resource_ip, backend_port_id, port, weight, administrative state
HealthMonitorSpec	type, interval, timeout, retries, HTTP path/expected response when supported

8. Logical IDs, Naming and Determinism

Logical IDs are controller-internal identities used to connect the graph and correlate desired and observed resources. Provider names are separately sanitized. Neither replaces provider external IDs returned by CMP.

Resource	Logical-ID input example
LB Service	clusterUID/sourceKind/sourceUID/sharedGroup
VIP	stackKey/frontend-address-family
Virtual Server	stackKey/protocol/frontendPort
Pool	stackKey/backendServiceUID/backendPort/routing-scope
Member	poolLogicalID/nodeUID/nodePort
Routing Rule	virtualServerLogicalID/host/path/pathType/priority
Monitor	poolLogicalID/monitor-fingerprint

```
logicalID = SHA256(canonical components) shortened only for display
providerName = sanitize(readable prefix + short hash)
externalID = value returned by CMP; never derived
```

9. Desired and Observed State Separation

Desired state	Observed state
Built exclusively from Kubernetes intent, controller configuration and authoritative identity resolution	Read from CMP APIs and Kubernetes status/annotations used solely as hints
Contains logical references	Contains external provider IDs and provisioning status
May request creation/update/deletion	Never directly changes desired semantics
Deterministic for same inputs	May be temporarily incomplete because CMP is asynchronous

The deployment layer compares normalized specifications, not raw JSON. Server-generated fields, timestamps, request IDs and provider defaults are removed or canonicalized before equality decisions.

9.1 Plan operations

Operation	Condition
Create	Desired resource exists; no owned observed resource
NoOp	Normalized desired equals observed and resource ready
Update	Mutable fields differ
Replace	Immutable field differs and safe replacement strategy exists
Delete	Owned observed resource absent from desired
Wait	Desired exists but dependency/provider resource is provisioning
Reject	Observed name matches but ownership does not

10. Dependency Graph and Ordering

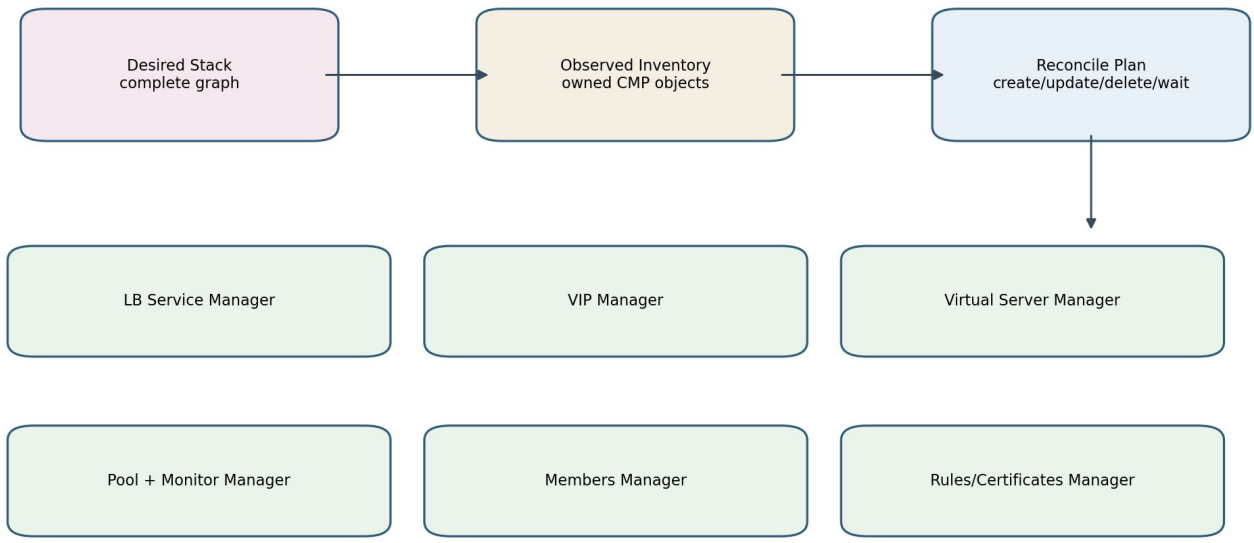

Figure 2-6. Deployment architecture and resource managers

10.1 Required creation dependencies

Resource	Required dependencies
LB Service	none
VIP	LB Service ready enough for VIP operation
Virtual Server	LB Service and VIP allocated/ready
Pool	Virtual Server exists
Health Monitor	Pool exists
Backend Member	Pool exists; node/VM/network identities resolved
Routing Rule	Virtual Server and referenced pool exist
Certificate attachment	LB Service/Virtual Server exists; certificate resource available

10.2 Deletion rules

```
Routing rules / certificate attachments
    -> members and monitors
    -> pools
    -> virtual servers
    -> VIP
    -> LB service
```

The exact ordering is derived from CMP endpoint relationships. A manager may skip a delete that CMP performs transitively, but the controller must verify absence before finalizer removal. Shared roots are protected by reference/ownership checks.

11. Deployment Manager Contracts

11.1 StackDeployer

```
type StackDeployer interface {
    Deploy(ctx context.Context, desired model.Stack) (DeploymentResult, error)
}

type DeploymentResult struct {
    VIPs       []ObservedVIP
    Resources  map[string]ObservedResource
    Ready      bool
    RequeueAfter time.Duration
}
```

The deployer is an orchestrator. It obtains an owned inventory, asks each manager to reconcile its resource type in dependency order, maintains a logical-to-external ID binding map, and returns readiness/status observations. It does not understand Service or Ingress fields.

11.2 Generic manager contract

```
type ResourceManager[TDesired any, TObserved any] interface {
    Discover(ctx context.Context, scope Scope, owner Ownership) ([]TObserved, error)
    Reconcile(ctx context.Context, desired []TDesired, observed []TObserved, bindings Bindings)
(ManagerResult, error)
}
```

Manager	CMP API family
LBServiceManager	.../load-balancers/lb_service
VIPManager	.../lb_service/{id}/vip
VirtualServerManager	.../{lb_service_id}/virtual-servers
PoolManager	.../virtual-servers/{vs_id}/pools
MemberManager	.../pools/{pool_id}/members
MonitorManager	.../pools/{pool_id}/monitors
RoutingRuleManager	.../virtual-servers/{vs_id}/routing-rules
CertificateManager	certificate APIs and LB/VS attachment APIs

12. Backend Resolution Architecture

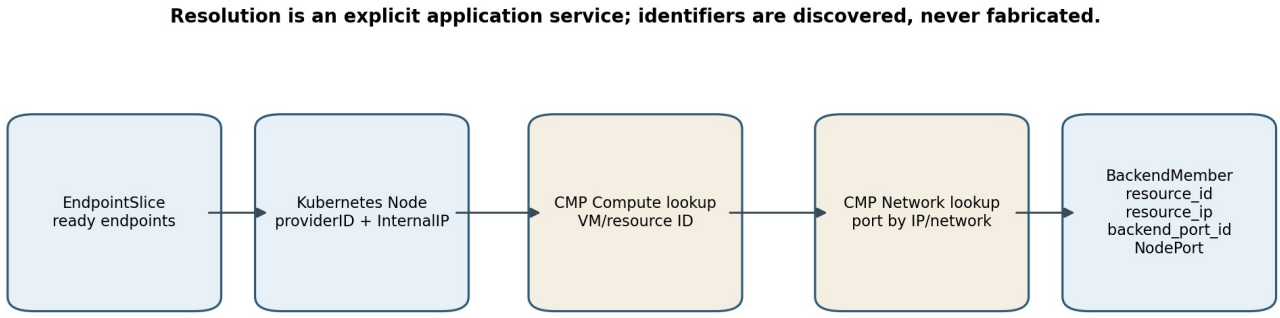

Figure 2-7. Node to CMP backend identity resolution

12.1 Resolution input and output

```
type BackendCandidate struct {
    NodeName    string
    NodeUID     types.UID
    InternalIP  net.IP
    ProviderID  string
    NodePort    int32
    ReadyEndpoints int
}

type ResolvedBackend struct {
    ResourceID   string
    ResourceType string
    ResourceIP   string
    BackendPortID string
    Port         int32
    Weight       int32
}
```

12.2 Resolution algorithm

```
1. Read ready EndpointSlice endpoints for the backend Service.
2. Map endpoint.nodeName to Node and apply readiness/traffic-policy eligibility.
3. Select Node InternalIP and parse providerID when present.
4. Query CMP compute inventory to resolve authoritative VM/resource ID.
5. Query CMP network port search-by-IP or network port inventory.
6. Require exactly one eligible port in the configured Shoot network.
7. Construct ResolvedBackend with provider-returned IDs and NodePort.
8. Cache positive immutable mappings for a bounded period; invalidate on Node/network changes.
```

12.3 Ambiguity and safety rules

- Zero matching VM or port is a dependency-resolution error; no placeholder member is created.
- Multiple matching ports are rejected unless network/project constraints reduce the result to one.
- The resolved port must belong to the configured organization, project and network.
- A member deletion removes pool membership only. It never invokes compute or port deletion APIs.
- Backend weight calculation is a builder policy; API serialization is a client concern.

13. Ownership, Discovery and Adoption

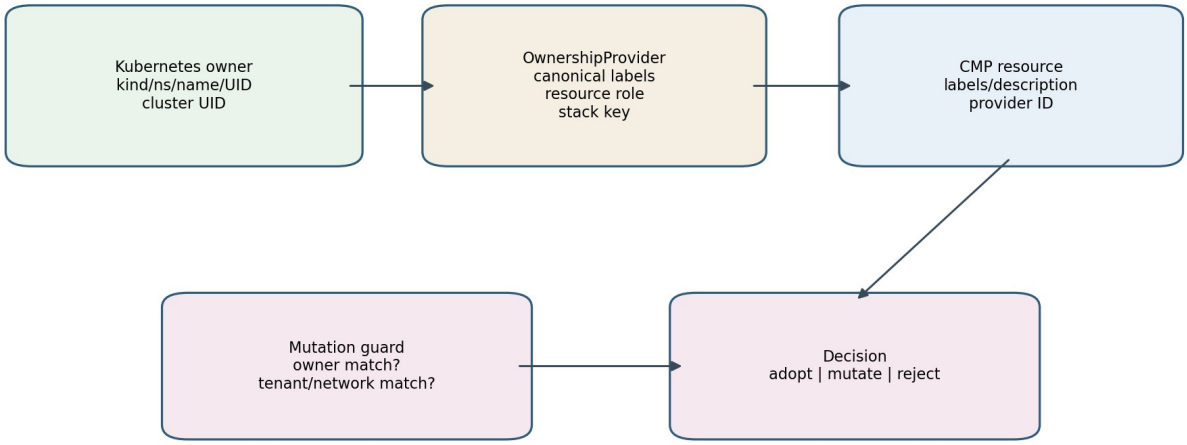

Figure 2-8. Ownership creation and mutation guard

13.1 Ownership schema

Field	Purpose
managed-by	Stable controller identity
cluster-uid	Shoot/cluster isolation
source-kind	Service, IngressGroup or future Gateway
source-namespace/name	Human diagnostics
source-uid	Prevents accidental adoption after name reuse
stack-key	Groups related provider objects
resource-role	lb-service, vip, virtual-server, pool, member, monitor, rule, certificate
shared-group	Explicit shared frontend identity when enabled
model-version	Allows controlled evolution of naming/model semantics

13.2 Discovery hierarchy

1. Use known external ID when available, then verify ownership.
2. List/filter by CMP labels where the API supports it.
3. Fall back to deterministic name plus full ownership verification.
4. Never mutate or delete a name match with absent/conflicting ownership.

13.3 Adoption policy

Observed resource	Action
Exact ownership match and compatible scope	Adopt/reconcile
Same source UID but older model version	Adopt through explicit migration rule
Same name, no ownership	Reject and surface conflict
Different source UID	Do not adopt; preserve resource
Owned but absent from desired	Delete in dependency-safe cleanup

14. Finalizer Architecture

14.1 Service finalizer

The Service finalizer is added before any provider create. On deletion or claim loss, the controller deploys an empty stack for that ownership scope, verifies required external resources are absent, removes generated Kubernetes objects, then removes the finalizer.

14.2 Ingress group finalizers

Ingress cleanup is group-aware. Active members retain the group frontend. Inactive members may have their finalizer removed only after resources owned exclusively by those members are absent and shared group resources remain correctly referenced by active members.

14.3 Finalizer invariants

- A provider API timeout never causes finalizer removal.
- A 404 counts as absent only for the exact scoped resource.
- Ownership conflicts block destructive cleanup and produce an operator-visible event.
- Emergency force-removal is an operational procedure outside normal reconciliation and must be audited.

15. Validation Pipeline

Stage	Examples	Output
Claim validation	class/loadBalancerClass, namespace policy	claimed or ignored
Kubernetes shape	ports, protocols, backend references, path types	typed normalized input
Feature compatibility	shared VIP, TLS, persistence, health monitors	supported or explicit permanent error
Provider context	organization, project, network, flavor	validated provider scope
Cross-resource validation	unique frontend tuples; route priority; compatible pool action	complete build plan
Resolved identity validation	VM and port belong to expected network	safe backend members

Validation errors are user-facing and stable. They include field paths and supported alternatives. Provider calls are not made for a model that fails validation, except read-only capability or identity checks required to validate it.

16. Resource Lifecycle State Model

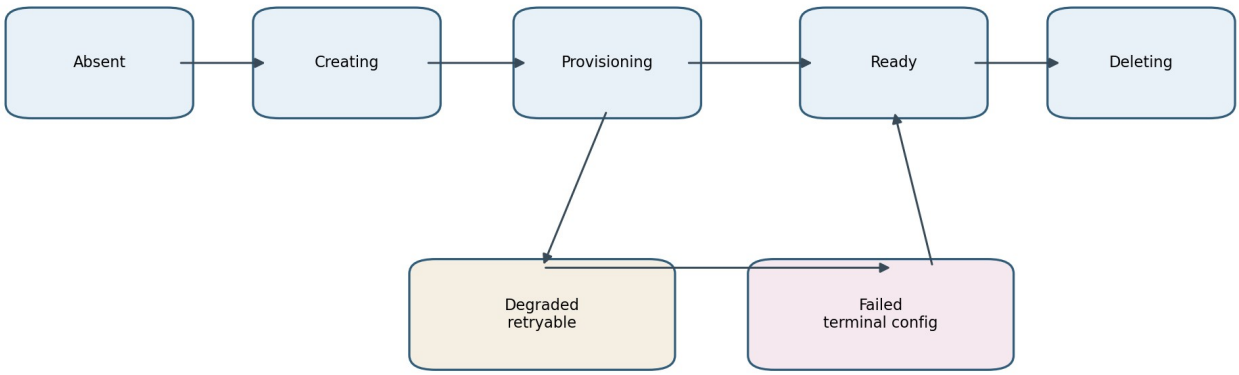

Figure 2-9. Provider resource lifecycle state model

State	Meaning	Controller action
Absent	No owned provider object	Create when desired
Creating	Create accepted; identity may or may not be returned	Rediscover/poll with bounded interval
Provisioning	Object exists but not ready	Do not publish dependent readiness
Ready	Desired and observed are converged	NoOp; publish status
Degraded	Retryable provider/dependency failure	Retry with classified backoff
Failed	Permanent desired configuration/provider rejection	Event and wait for desired change
Deleting	Delete accepted; object still visible	Poll until absent

17. Typed CMP Client Architecture

17.1 Client boundaries

Client	Representative operations
LBServiceClient	List/Get/Create/Delete LB services; labels
VIPClient	List/Create/Delete VIP; obtain allocated address
VirtualServerClient	List/Get/Create/Patch/Delete; health check
PoolClient	Create/Get/Delete; set default
MemberClient	Create/Update/Delete; administrative state; health
MonitorClient	List/Create/Update/Patch/Delete
RoutingRuleClient	List/Create/Delete rules
CertificateClient	Upload/list/delete and attach to LB/virtual server
ComputeClient	Resolve VM/resource by provider identity or IP
NetworkClient	Search port by IP; get port/network metadata

17.2 Transport contract

- All methods accept context and honor cancellation/timeouts.
- Authentication token acquisition/refresh is isolated from resource clients.
- Requests and responses use generated or hand-maintained typed DTOs; raw map[string]any does not escape the client package.
- HTTP status, provider error body, request ID and retry hints are converted to typed errors.
- Sensitive headers and certificate material are redacted from logs.
- Retries at transport level are limited to safe idempotent operations or requests with a provider-supported idempotency key.

```
type APIError struct {
    Operation string
    StatusCode int
    RequestID string
    RetryAfter time.Duration
    Category ErrorCategory
    Message string
}
```

18. Status, Conditions, Events and Readiness

Output	Rule
Service.status.loadBalancer.ingress	Write allocated VIP after frontend readiness gate
Ingress.status.loadBalancer.ingress	Write group VIP after required HTTP/HTTPS frontends are ready
Events	Emit accepted/provisioning/ready and actionable failures, rate-limited
Conditions	Use extension-owned condition surface only if a suitable API exists; otherwise Events and metrics
Observed IDs	Store only as optimization hints in a controlled annotation/status object if required; always verify ownership

18.1 Recommended readiness gate

A frontend is ready when the LB Service, VIP and required Virtual Server are provider-ready, and each required default/routed pool exists. Backend health may be reported as degraded rather than blocking VIP publication, because an application can temporarily have zero ready endpoints. This policy is finalized per Service and Ingress semantics in Parts 3 and 4.

19. Concurrency, Idempotency and Restart Recovery

- Leader election ensures one active mutating controller instance per Shoot.
- Per-key controller-runtime serialization is relied upon, but provider operations remain idempotent because duplicate events and failover can occur.
- Create-time uncertainty is handled by rediscovery through ownership before issuing another create.
- No correctness-critical state exists only in memory.
- Logical IDs and ownership survive restarts; external IDs are rediscovered.
- Status writes use merge/optimistic patches and do not overwrite unrelated fields.
- Caches improve performance but may be discarded without changing correctness.

20. Package Architecture

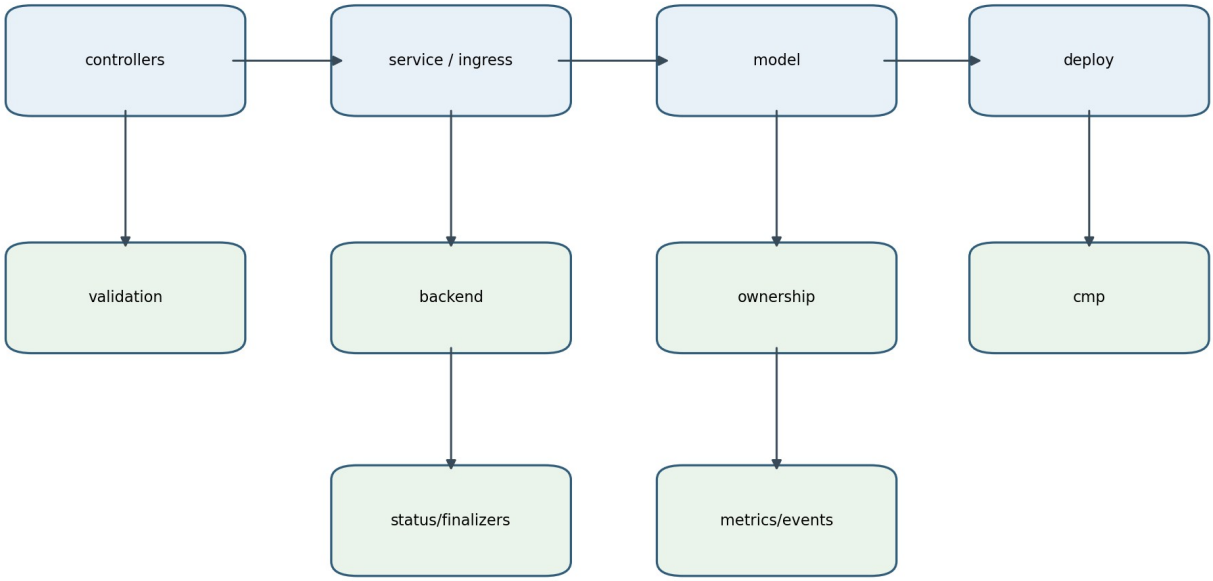

Figure 2-10. Proposed package dependency structure

Package	Contents
cmd/controller-manager	flags, configuration, dependency wiring, manager start
controllers/service	Service reconciler and event mapping
controllers/ingress	Ingress-group reconciler and event mapping
pkg/service	Service normalized input and model builder
pkg/ingress	Ingress grouping, normalized input and model builder
pkg/model	provider-independent internal graph and resource specs
pkg/deploy	stack deployer, planner, bindings and manager interfaces
pkg/deploy/lbservice etc.	resource-specific CMP reconciliation managers
pkg/cmp	typed clients, DTOs, auth, transport and errors
pkg/backend	EndpointSlice/node selection and CMP identity resolution
pkg/validation	shared and feature-specific validation
pkg/ownership	ownership envelope, naming, discovery filters
pkg/finalizers	Service and group finalizer helpers
pkg/status	Service/Ingress patch writers
pkg/metrics and pkg/events	observability contracts

20.1 Dependency rules

```
controllers -> service/ingress builders, deploy, status, finalizers
service/ingress -> model, validation, backend, ownership
deploy -> model, cmp, ownership
backend -> Kubernetes readers + compute/network clients
cmp -> transport/auth only
model -> no Kubernetes clients and no CMP clients
```

21. Key Go Interfaces

```
type ServiceModelBuilder interface {
    Build(ctx context.Context, svc *corev1.Service) (model.Stack, BuildResult, error)
}

type IngressModelBuilder interface {
    Build(ctx context.Context, group IngressGroup) (model.Stack, BuildResult, error)
}

type BackendResolver interface {
    Resolve(ctx context.Context, request BackendResolutionRequest) ([]ResolvedBackend, error)
}

type OwnershipProvider interface {
    ForObject(cluster ClusterIdentity, obj client.Object, role model.ResourceRole, stackKey string)
model.Ownership
    Matches(expected model.Ownership, observed cmp.Labels) bool
}

type StatusWriter interface {
    WriteService(ctx context.Context, svc *corev1.Service, obs FrontendObservation) error
    WriteIngresses(ctx context.Context, group IngressGroup, obs FrontendObservation) error
}
```

22. Architecture Decision Records

ADR	Decision	Rationale
ADR-02-01	Use separate Service and Ingress-group reconcilers	Ownership, watch graph and semantics differ
ADR-02-02	Build a complete internal stack before mutation	Enables validation, deterministic diff, testing and shared deployment
ADR-02-03	Use resource-specific deployment managers	CMP resources have different endpoints, dependencies and mutability
ADR-02-04	Use node/VM plus NodePort target mode initially	Matches CMP member identity needs and current reachable network model
ADR-02-05	Make ownership first-class	Safe adoption, update and deletion cannot rely on names
ADR-02-06	Do not persist internal model as CRDs	Kubernetes intent and CMP state are sufficient sources of truth
ADR-02-07	Treat current F5 code as integration reference only	Avoid inheriting unverified architecture
ADR-02-08	Use asynchronous readiness rather than synchronous long polling	Keeps reconciliation bounded and resilient
ADR-02-09	Share deployment/client foundations between Service and Ingress	Same provider resource hierarchy; avoids divergent behavior
ADR-02-10	Introduce shared VIP only through explicit group identity	Prevents accidental cross-owner coupling and unsafe deletion

23. Implementation Boundaries for Incremental Delivery

The target architecture does not require a big-bang rewrite. It should be introduced through small vertical slices while preserving working CMP integration.

Slice	Deliverable	Exit criterion
1	Typed ownership and naming package	Unit-tested canonical metadata and matching
2	Typed CMP client facade around existing HTTP integration	No controller uses raw HTTP/JSON
3	Internal model primitives and LBService/VIP manager	Create/read/delete idempotency verified
4	Backend resolver with compute/network port lookup	Node -> VM -> backend_port_id integration test
5	Service model builder for one TCP port	Desired stack unit tests
6	Stack deployer and Service controller integration	Single-port Service converges and cleans up
7	Pool/member diff; multi-port support	Scale and node replacement do not recreate frontend
8	Ingress builder and routing rules	Host/path routing verified against CMP
9	Certificates, monitors and advanced policy	Capability-gated features
10	Migration/adoption and production hardening	Restart, duplicate event, partial failure and HA tests

24. Design Invariants

1. A controller restart does not lose ownership or require in-memory recovery state.
2. No CMP resource is mutated or deleted unless ownership and provider scope are verified.
3. Backend resource IDs and backend_port_id values are returned by CMP lookup APIs, never synthesized.
4. A member removal cannot delete a VM or network port.
5. An EndpointSlice or node change reconciles members without recreating unchanged VIPs or Virtual Servers.
6. Service and Ingress builders share model/deployment contracts but not Kubernetes-specific semantics.
7. Raw CMP JSON and HTTP status handling remain inside pkg/cmp.
8. A finalizer is removed only after mandatory owned external resources are verified absent.
9. Logical IDs are deterministic and independent from provider-generated IDs.
10. Unsupported Kubernetes semantics fail explicitly before provider mutation.
11. Every provider call is context-bound, time-limited and observable.
12. The desired-state model remains acyclic and complete for the reconciliation key.

25. Part 2 Completion and Handoff

This document establishes the fixed component and internal-model foundation. Part 3 applies these contracts to the complete Service type=LoadBalancer design, including claim rules, port expansion, traffic policy, endpoint/node eligibility, backend weighting, status publication, update behavior, deletion, and Service-specific sequence/activity diagrams. Part 4 then applies the same architecture to Ingress grouping, routing rules and TLS.

Appendix A - CMP Endpoint Mapping Used by the Architecture

Model resource	CMP endpoint pattern
LB Service	/load-balancers/.../load-balancers/lb_service/
VIP	.../lb_service/{lb_service_id}/vip
Virtual Server	.../load-balancers/{lb_service_id}/virtual-servers
Routing Rule	.../lb_service/{lb_service_id}/virtual-servers/{virtual_server_id}/routing-rules
Pool	.../virtual-servers/{virtual_server_id}/pools
Member	.../pools/{virtual_server_pool_id}/members
Monitor	.../pools/{virtual_server_pool_id}/monitors
Default pool	.../pools/{virtual_server_pool_id}/set-default
Virtual-server certificates	.../virtual-servers/{virtual_server_id}/certificates/
Network port lookup	/networks/.../networks/ports/search-by-ip/ and network port inventory
Certificate inventory	/certificates/domain/{organization_id}/project/{project_id}/certificates/

Appendix B - Traceability to Previous LLD

The following valid concepts from the previous LLD are retained and refined: thin controllers, builders, desired-state graph, typed clients, ownership, finalizers, backend selection, Node-to-VM mapping, backend_port_id, LB Service, VIP, Virtual Server, pools, members, monitors, routing rules, status, idempotency, deletion ordering, restart recovery, observability, security and incremental implementation. Statements about the correctness or limitations of the current implementation are intentionally excluded from the target design.